

When Do Music Graphs Matter?

A Four-Task Study of Text, Structure, Fusion, and Cross-Modal Retrieval

Anonymous Author
Independent research report

Abstract—Music understanding systems often combine text and audio, and audio is often represented as a graph: short segments become nodes, with edges linking segments that are close in time or similar in content. When a graph’s structure is a deterministic function of its own node features, the data-processing inequality guarantees it cannot add Shannon information beyond those features – an analytic fact, not something an experiment can test. What experiments can test is the practical question: whether such a graph still helps a real, finite model, for instance as a computational shortcut or inductive bias. We test that practical question directly, across four tasks built on the same node features and graph construction rule: text-only tag prediction (no graph at all), graph-based genre classification, graph-plus-text fusion, and audio-text contrastive retrieval. Tasks 2 and 4 each compare their graph-based model against a width/depth-matched, lower-parameter, edge-free alternative on held-out test data; Task 3’s own direct comparison is validation-only, so its evidence instead comes from two test-set diagnostics: branch-zeroing inside the trained fusion model, and a cross-tabulation of independently-trained single-branch checkpoints. No task shows a graph-based model with a *learned* aggregation step clearly and reliably ahead of its edge-free counterpart. One result complicates this and is reported rather than hidden: a non-learned exact 2-hop neighbor aggregation in genre classification outperforms GraphSAGE and both edge-free readings (though not the CNN baseline, which uses neither the graph nor these node features) – though it also uses a different training recipe (full-batch optimization, no dropout, a different epoch budget) than the models it is compared against, so this is suggestive of under-exploited topology, not a clean ablation. In genre classification, the pooling-matched comparison between the two *trained* models favors the edge-free encoder. In fusion, zeroing the graph branch inside the trained model changes only 1.7% of individual (example, tag) predictions and costs 0.009 macro-F1: a small positive contribution, not zero, just small. A separate complementarity check – comparing the independently-trained graph-only and text-only models – at first appears to show the graph-only model correcting 48.5% of the text-only model’s errors, but splitting these by label polarity shows the graph-only model predicts a tag present at all for only 5 of 50 tags; fewer than 1% of its apparent corrections are it actually detecting a tag the text model missed, and the rest are it defaulting to “absent” while text wrongly said “present” – this diagnostic, as originally framed, does not show real complementary signal. In retrieval, a graph-based and a no-graph encoder show no reliable difference across three training seeds, on every reported metric. We also checked two alternative explanations rather than assume them away: caption text leaking target labels directly (it does, substantially), and duplicate audio between train and test in one dataset (real, confirmed for a small number of tracks, though its actual effect on any score was not separately measured, and is left open as an unresolved caveat for the smallest comparison

specifically). We do not conclude that graphs are useless for music – only that this specific similarity-based construction does not reliably help beyond its own node features, for these tasks, on these datasets. We report this as the paper’s central finding, not as a failed architecture search.

I. INTRODUCTION

Music understanding systems increasingly combine several kinds of input: a natural-language description of a track, and the track’s own audio. Audio is often turned into a graph before it reaches a model – a track is split into short segments, each segment becomes a node, and edges connect segments that are close in time or acoustically similar. This is a reasonable design choice: music has real relational structure (repetition, transition, similarity between sections), and a graph neural network (GNN) can in principle use that structure. Similarity between short audio segments, computed from acoustic features and thresholded or visualized as a matrix, is a long-established technique for finding structure within a single track [1]; this paper’s graph construction applies the same underlying idea (pairwise segment similarity) to build a graph rather than a novelty curve. The message-passing GNN family this paper draws its encoder from includes several related architectures beyond the one used here – GCN [2], GAT [3], and GIN [4], in addition to GraphSAGE [5]. Separately, a recognized concern in the broader GNN literature is that graph-based models are not always compared against simpler baselines under matched, controlled conditions [6], [7] – this paper takes that concern as a direct methodological starting point, not only a citation.

But if the edges are built directly from the same features already given to each node – for instance, connecting two segments whenever their feature similarity crosses a threshold – then the graph’s structure is, in the strict information-theoretic sense, a restatement of information already present in the node features, not a new observation about the track. This does not automatically mean such a graph is useless: a deterministic function of the features could still act as a useful computational shortcut or inductive bias, easing optimization even without adding information a sufficiently powerful model could not already extract from the features alone. Whether either of these possibilities – added information, or a useful-but-informationless shortcut – actually helps in practice, for a specific graph construction and a specific set of tasks, is

an empirical question, not something to assume from the architecture diagram alone.

This paper answers that question for one concrete system, across four tasks that share the same node-feature definition and the same similarity-based edge construction rule – though not, as Section III discloses precisely, the same audio segmentation length or dataset split in every task: a text-only tag classifier (Task 1, no audio at all), a graph-based genre classifier compared against an edge-free alternative (Task 2), a graph-plus-text fusion model (Task 3), and a contrastive audio-text retrieval model (Task 4). Each task, on its own, answers a narrower question. Together, they form a single experimental ladder:

- **Task 1** asks how much target information language alone already provides, before any audio or graph is involved.
- **Task 2** asks whether the graph’s topology helps audio-only classification beyond what the same node features give a structure-free model.
- **Task 3** asks whether the graph adds information that a strong text model does not already have, once both are available together.
- **Task 4** asks whether any graph benefit found in classification or fusion survives under a different objective entirely: matching audio to text by similarity, with no genre or tag labels – only audio-caption pairing as supervision.

We do not set out to show that graphs are useless for music, and we do not find that. We set out to check, directly and with matched controls wherever they could be arranged – Section V discloses the specific exceptions found on inspection, including one confound in Task 2’s own training scripts – whether *this* similarity-based graph construction earns its place in *these* tasks. Task 1 has no graph at all and serves as the anchor; Tasks 2–4 each include a direct graph-vs-edge-free comparison, and in none of the three does the graph-based model come out clearly and reliably ahead. Task 2’s fairest (pooling-matched) comparison favors the edge-free alternative, though a non-learned 2-hop control complicates this (Section VIII). Task 3’s single standalone comparison is mixed and inconclusive, but its two follow-up diagnostics – branch-zeroing (an inference-time intervention on the trained fusion model, finding it relies on the graph branch only slightly) and complementarity (correlational, comparing two separately-trained single-branch models, finding almost no positive-label rescue by the graph-only model) – together find the graph contributing little the fusion model actually ends up relying on. Task 4 shows no consistent advantage either way across three seeds. We report this consistency, with each result’s own caveats, as the paper’s main finding. We also check two alternative explanations that could produce this pattern without the graph itself being at fault – caption text directly leaking target labels, and duplicate or leaked audio inflating a model’s apparent score – and report what we find, rather than assume either one away.

II. FOUR-TASK STUDY DESIGN

The four tasks are not identical in dataset, label space, objective, or metric – Task 1 has no graph at all, Task 2

is single-label classification on GTZAN, Task 3 is multi-label classification on MusicCaps, and Task 4 is a ranking objective with no genre or tag class labels, using audio-caption pair identity as supervision. What they do share is the node-feature definition (32-dimensional chroma plus MFCC), the same similarity-based graph construction rule, and closely matched encoder architectures where a graph is involved; Task 4 was also built in a separate implementation with a different segment length and dataset split (Section III discloses this precisely). This shared representation is what makes a cross-task comparison informative at all: each task’s own graph-vs-edge-free result is its own self-contained comparison, with its own explicitly disclosed matching exceptions (Section V) rather than a uniform fairness guarantee across tasks, and it is the *consistency of the overall conclusion* those results support – not an identical numeric comparison in every task, and not an assumption that every task used byte-identical inputs – that the cross-task analysis in Section VII relies on.

Task 1 – Text Tag Prediction. A text-only reference point. A BERT-family encoder predicts tags from a caption alone, with no audio and no graph at all. This establishes how much of the target signal language already carries by itself – a qualitative anchor for the rest of the paper’s “does the graph add anything” question, not a numerical baseline every later task’s own metric is scored against, since Task 2 uses a different dataset and label space and Task 4 uses a different metric entirely.

Task 2 – Graph-Based Audio Classification. A graph neural network predicts genre from an audio-only segment graph, compared against (a) a non-graph CNN baseline operating directly on the log-mel spectrogram, and (b) a width/depth-matched, lower-parameter, edge-free encoder using the exact same node features as the graph model, with every edge removed. Comparison (b) tests whether the graph encoder, as a whole, adds anything over a structure-free alternative; because it changes the aggregation operator along with the edges, it does not isolate topology alone – Section VI-B also reports a same-architecture, edges-only intervention from a companion audit for that narrower question.

Task 3 – Multimodal Context Prediction. A graph encoder of the same architecture as Task 2’s is fused with a separately initialized instance of the same pretrained text-encoder architecture used in Task 1 (not Task 1’s own trained weights), compared across four fusion strategies (text-only, graph-only, concatenation, cross-attention) plus two diagnostics: zeroing the graph branch inside a trained fusion model, and cross-tabulating which examples the text-only and graph-only models get right or wrong. This tests whether the graph is complementary to text, not just whether it is useful on its own.

Task 4 – Audio-Text Retrieval. A contrastive dual encoder learns to match a caption to its audio clip, with no genre or tag class labels, using only audio-caption pairing as supervision – a different objective from classification, scored by ranking (Recall@K) rather than accuracy. The same graph-vs-edge-free comparison from Task 2 is repeated here, to test whether

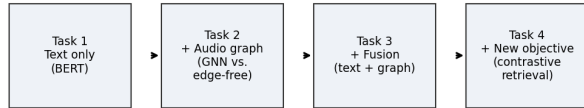


Fig. 1. The four-task progression. Each task keeps the same node features and graph construction rule, and adds one new element: audio (Task 2), fusion with text (Task 3), or a different objective (Task 4).

any graph advantage (or its absence) generalizes beyond classification to a fundamentally different task type.

Figure 1 summarizes the ladder: each task adds one element – audio, fusion, then a different objective – while keeping the node-feature definition and graph construction rule fixed (Section III discloses where the underlying implementation still differs between tasks).

III. DATA AND REPRESENTATIONS

A. Datasets and splits

Two datasets are used. **GTZAN** [8] (Task 2): 10 genres, 999 usable 30-second clips (699/150/150 train/val/test) – a fixed, study-specific, genre-stratified split generated with a documented seed, not an official GTZAN benchmark split, used throughout this study. **MusicCaps** [9] (Tasks 1, 3, 4): natural-language captions paired with short tag phrases, used as a text-only corpus (Task 1) and, where audio is also available, as caption-plus-audio-graph pairs (Tasks 3, 4) – these are two different subsets of MusicCaps, not one, since Task 1 needs no audio and so can use every caption with usable text regardless of whether that clip’s audio was ever downloaded, while Tasks 3/4 are restricted to the smaller subset with a cached audio graph. Task 1 uses 3,523/755/756 captions (756 test); the audio-paired subset used by Tasks 3 is 3,287/704/715.

An important disclosure. Tasks 1–3 share one implementation, though not one split or one segment length throughout: Task 1 uses the caption-only 3,523/755/756 split above and no audio segmentation at all; Task 2 uses GTZAN with 2-second segments; Task 3 uses MusicCaps with 1-second segments and the audio-paired 3,287/704/715 split (50-tag vocabulary). Task 4 was built and run separately, in a second, independent implementation of the same architecture family (2-second MusicCaps segments, a 3,136/673/673 split obtained by merging in additional cached audio, and a 30-tag vocabulary for its own zero-shot sanity check). Both implementations share the same node-feature definition, similarity-graph rule, and encoder architectures (Section IV), and the paper’s cross-task argument (Section VII) does not depend on the exact segment length or split size being identical – but the two are not byte-for-byte the same pipeline, and we say so plainly rather than imply a single shared codebase produced all four tasks’ numbers.

B. Audio segmentation and node features

Task 1 uses no audio at all – text only. Where audio is used (Tasks 2–4), each track is split into non-overlapping segments: 2 seconds for GTZAN (Task 2) and for Task 4’s MusicCaps pipeline; 1 second for Task 3’s MusicCaps pipeline (see the disclosure above). Each segment’s node feature vector is 12-dimensional chroma concatenated with 20-dimensional MFCC (32 dimensions total) in every audio pipeline, a purely local, per-segment summary with no cross-segment information built in.

C. Graph construction

Edges connect (a) temporally adjacent segments, and (b) any pair of segments whose feature cosine similarity exceeds a fixed threshold $\tau = 0.8$ – the same underlying pairwise-segment-similarity computation used in self-similarity-matrix analysis of audio structure [1], here thresholded into a graph’s edge set rather than read off as a novelty curve. Writing $X = \{x_1, \dots, x_n\}$ for a track’s segment features, the similarity edge set is

$$A_{\text{sim}} = \{(i, j) : \cos(x_i, x_j) > \tau\},$$

a deterministic function of X alone – a property that matters directly for interpreting the results in Section VII. This scoping applies to the similarity edges specifically, not to the deployed construction’s temporal-adjacency edges, which encode segment order – information not recoverable from an unordered feature set, and therefore outside this argument’s premise. Measuring the deployed construction directly confirms it is dense: mean edge density 95.4% (MusicCaps) to 97.9% (GTZAN), with 76.4% to 84.0% of individual graphs *fully* connected (a stricter, related statistic, not the same number; these two datasets are the ones this paper’s own comparisons use – a companion audit measures a third dataset, MagnaTagATune, not used here, at a comparably high 98.9% density and is not included in this range). The MusicCaps figure was measured on Task 3’s 715-example test split specifically (1-second segments, roughly 10 nodes per graph); it was not separately re-measured for Task 4’s independent 2-second-segment MusicCaps pipeline, so we report it as evidence about Task 3’s graphs and do not extend it to Task 4’s without a direct check. Randomly rewiring or resampling its edges reproduces almost the same graph under a different name. A repaired, sparser construction (mutual k -nearest-neighbor, $k \in \{4, 8, 16\}$, verified non-degenerate before use, though only three of six dataset/ k combinations pass this check – $k=16$ fails on both datasets and MusicCaps $k=8$ also fails) is used in a companion audit of this same system [10]; the present paper uses the original, denser construction throughout, since Task 2/3/4’s baseline checkpoints were trained on it, and notes this as a limitation (Section VIII).

D. Text encoding

Captions and tag phrases are tokenized with a standard BERT-family tokenizer and encoded with a fine-tuned BERT/DistilBERT model (Section IV).

E. Leakage and duplicate checks

We ran three checks before treating any result as evidence.

Caption-level label leakage (Task 1). MusicCaps captions were checked for direct lexical overlap with the target tag vocabulary: averaged across the 50 tags (a label-macro average, not a per-clip average), 69.1% of the time a tag’s own word or a near-literal match for it appears in the caption of a clip it labels. This is a real, substantial effect and is discussed directly in Section VII – it explains part of why text alone is so strong in Task 1, though a caption-sanitization control (target words removed, model retrained from scratch) still recovers 84.3% of original performance, showing the effect is real but not total.

GTZAN duplicate and split-leakage audit. We hashed all 1,000 raw GTZAN audio files (999 of which are used across the train/val/test splits) and found **14 exact, byte-identical duplicate pairs** – consistent with GTZAN’s documented quality issues [11]. Cross-checking these pairs against the train/val/test split used throughout this paper found **7 of the 14 pairs cross a split boundary** (e.g. one copy in train, the other in test), meaning a small number of test tracks have a byte-identical duplicate in the training set. One duplicate pair additionally carries two different genre labels (`metal.00058.wav` and `rock.00016.wav` are byte-identical but filed under different genres), a labeling inconsistency independent of splitting. In absolute terms this affects a small fraction of the 150-track test set (**3 of 150 test tracks** have an exact training-set duplicate). This is unlikely to explain Task 2’s largest gaps (the CNN-vs-GraphSAGE gap of 0.168, or the pooling-confounded edge-free comparison of 0.045), but we do not treat it as ruled out for the smaller, pooling-matched Task 2 contrast (0.0072) – see Section VII for why. It is real, previously undocumented for this specific split, and disclosed rather than corrected after the fact (correcting it would require retraining every Task 2 model on a new split, changing every number in this paper – left as a direct next step, not silently patched here).

MusicCaps split-leakage audit. All MusicCaps train/val/test splits were checked for duplicate or overlapping YouTube IDs, both within a split and across splits. None were found – 3,287/704/715 unique IDs with zero overlap. This check is ID-based; it would not catch two different clips of the same underlying recording, and audio-content-level near-duplicate detection for MusicCaps was not run (Section VIII).

Artist-level leakage. Neither dataset ships usable artist metadata (MusicCaps’ `author_id` field identifies the caption’s human annotator, not the track’s performing artist), so artist-level leakage could not be checked for either dataset. This is a genuine, undisclosed-by-necessity limitation, not a check that was skipped.

IV. MODELS AND OBJECTIVES

A. Task 1: text tag classifier

A DistilBERT encoder [12], a distilled version of BERT [13], with a linear multi-label head predicts a 50-tag vocabulary from a caption: $t = \text{BERT}_{\text{CLS}}(X_{\text{text}})$, $\hat{y}_k = \sigma(w_k^\top t + b_k)$, trained with per-tag binary cross-entropy.

B. Task 2: graph-based genre classifier and controls

A 2-layer GraphSAGE encoder [5] with mean-pool readout classifies GTZAN [8] genre, with each non-final layer updating node representations as

$$h_i^{(l+1)} = \sigma(W^{(l)} \cdot \text{CONCAT}(h_i^{(l)}, \text{MEAN}_{j \in \mathcal{N}(i)} h_j^{(l)})),$$

(the implementation applies σ and dropout between layers, not after the final one, omitted above for brevity) before mean-pool readout $g = \frac{1}{|V|} \sum_i h_i^{(L)}$. Three controls probe what the graph branch contributes, though none isolates topology cleanly on its own (Section V discloses each control’s specific confound): a CNN baseline, following the standard convolutional auto-tagging approach for spectrogram-based audio classification [14], [15], with no graph and no shared features at all; an edge-free set encoder that replaces every GraphSAGE layer with a per-node MLP of identical width and depth, keeping the same node features and readout but removing every edge (changes the aggregation operator along with topology); and a non-learned exact 2-hop neighbor aggregation, which uses the same topology as GraphSAGE and no trained message-passing function, but also a different training recipe for its classifier head (Section VIII). The cleaner topology-only intervention this paper relies on for that narrower question is a same-architecture real/shuffled/random-edge factorial from a companion audit [10] (Section VI-B), which changes edges alone.

C. Task 3: fusion strategies and diagnostics

The same GNN and BERT component *architectures* as Tasks 1 and 2 (not their trained weights, which are separate, freshly-initialized instances for Task 3) are combined four ways: *bert_only* (text alone), *gnn_only* (graph alone), *concat* ($z = \text{CONCAT}(g, t)$), and *cross_attention* ($A = \text{softmax}(QK^\top/\sqrt{d})$, $Q = gW_Q$, $K = H_{\text{text}}W_K$, $V = H_{\text{text}}W_V$, $z = \text{CONCAT}(g, AV)$; the implementation also adds a padding mask to the attention logits $QK^\top/\sqrt{d}$ before the softmax that produces A , omitted above for brevity, so padded caption tokens receive zero attention weight). Two diagnostics go beyond the four-way comparison itself. **Branch zeroing:** inside a trained *concat* model, the graph embedding g is replaced with zeros at evaluation time, with no retraining, isolating what the graph branch contributes conditional on text already being present. **Complementarity cross-tabulation:** per-(example, tag) predictions from the trained *bert_only* and *gnn_only* checkpoints are compared directly, to test whether the graph model is right on a distinct subset of cases the text model gets wrong, rather than only comparing aggregate scores.

D. Task 4: contrastive dual encoder and controls

A CLIP-style [16] dual encoder pairs the same GNN and BERT components under a symmetric InfoNCE objective [17]. The same two-tower, contrastive design underlies music-specific audio-text systems – MuLan [18] and the contrastive audio-language model of Manco et al. [19] – as well as general-purpose audio-language systems not specific to music,

such as CLAP [20]; the same objective is used for text-to-music retrieval specifically by Doh et al. [21]; this task follows that same general design at a much smaller scale, with the two towers built from this paper’s own GNN and BERT components rather than the larger pretrained encoders those systems use – the implementation averages this audio-to-text direction with the text-to-audio direction (swapping which side is summed over in the denominator), giving the loss $\mathcal{L} = \frac{1}{2}(\mathcal{L}_{a2t} + \mathcal{L}_{t2a})$ its “symmetric” name; one direction is shown here for brevity:

$$\mathcal{L}_{a2t} = -\frac{1}{N} \sum_i \log \frac{\exp(\text{sim}(g_i, t_i)/\tau)}{\sum_j \exp(\text{sim}(g_i, t_j)/\tau)}.$$

The same graph-vs-edge-free comparison from Task 2 is repeated here: an otherwise identical dual encoder with the GNN branch replaced by a width/depth-matched, lower-parameter, edge-free MLP encoder, trained and evaluated identically.

V. EXPERIMENTAL PROTOCOL

Metrics. Classification tasks (1, 2, 3) use macro-F1 as the primary metric throughout. Micro-F1 is also available for Task 1 and Task 2’s main models, and AUC-PR for Task 1 and Task 3; Task 2’s edge-free and 2-hop controls, evaluated for macro-F1 only, do not have micro-F1/AUC-PR recorded, so this paper does not report them for every model in every table – only where the underlying evaluation produced them. Retrieval (Task 4) reports Recall@{1,5,10} in both directions (audio→text and text→audio), plus a zero-shot tag-prediction check (tag names embedded as text, no supervised tag training) as a sanity check that the learned space carries semantic content at all.

Model selection. Every model evaluated on the held-out test set was selected by best validation-set score (macro-F1 for classification, average Recall@10 for retrieval) before that one test-set evaluation. Task 3’s edge-free *gnn_only* comparison (Section VI-C) is validation-only in a stricter sense than this: its training scripts run a fixed 20 epochs and report the peak validation macro-F1 observed across that run, rather than saving and restoring a selected best checkpoint – so “best” there describes the reported number, not a checkpoint carried forward to any further evaluation. This comparison is explicitly marked validation-only wherever it is reported, not presented as a test-set result.

Seeds. Task 2/3’s main comparisons in Sections VI-B–VI-C use a single training run each, matching the resources available for this study; this is a real limitation and is stated as such rather than implied to be more robust than it is. The deployed GraphSAGE checkpoints these comparisons are measured against were trained with a documented seed (42); the Task 2 exact 2-hop control script also fixes this same seed (42) for its Python/NumPy/PyTorch random state, so that comparison is seed-matched. The edge-free set-baseline scripts (Task 2 and Task 3) do not set or record an explicit seed, so those specific single-run contrasts are not seed-matched and are not guaranteed to be exactly reproducible from a fresh run – a further reason to read the edge-free numbers specifically

as indicative rather than precise. Task 4’s graph-vs-edge-free comparison, being the paper’s most direct causal claim, is repeated across three training seeds and reported as mean $\pm$ standard deviation (Section VI-D).

Fairness controls. Every graph-vs-edge-free comparison in this paper holds node features, optimizer, learning rate, batch size, epoch budget, and evaluation code fixed. One exception, found on inspection of the training scripts rather than assumed away: Task 2’s GraphSAGE training call does not pass its configured dropout value (0.4) through to the model, so it silently trains at the model class’s default of 0.2, while the edge-free encoder is passed the configured 0.4 explicitly – so this comparison is not dropout-matched as the paper’s earlier drafts assumed, on top of the aggregation-operator difference discussed next. We disclose this rather than silently correct it after the fact (correcting it would mean retraining GraphSAGE and changing every Task 2 number that depends on it); Section VII discusses what this does and does not imply for the smallest Task 2 gap specifically. Separately, replacing GraphSAGE layers with per-node MLP layers also removes the aggregation operator itself, not topology alone, so this comparison is an edge-free-encoder control, not a pure edge-ablation. Neither guarantee extends to Task 2’s non-learned 2-hop control, which is not an edge-free comparison at all (it keeps the same topology as GraphSAGE) and uses a different training recipe for its classifier head – full-batch optimization, no dropout, a fixed 200-epoch budget, versus GraphSAGE’s mini-batches and early stopping (both use the same nominal weight decay, 10^{-4} , so that specific setting is not a distinguishing factor; Section VIII discloses the recipe difference precisely and treats the 2-hop result accordingly, as exploratory rather than a controlled ablation). Section VI-B reports a second, cleaner topology-only intervention (real vs. shuffled vs. random edges on the same GraphSAGE architecture, from a companion audit [10]) that isolates edge identity alone and finds the same qualitative pattern. Parameter counts are reported for Task 2’s comparison (Table I); Task 3’s edge-free encoder is also smaller than its GraphSAGE counterpart (9,522 vs. 15,666 parameters, from the same companion evidence base [10]), though this paper’s own tables do not repeat that count for Task 3/4. Wherever it is reported, the edge-free alternative uses fewer parameters than its graph counterpart (Section VI), so no graph-vs-edge-free gap in this paper can be explained by the edge-free model having more capacity.

Class weighting. Weighting was not held fixed across every Task 3 number, and we say precisely which is which rather than imply otherwise. Table II’s *bert_only* (0.7298), *concat* (0.7061), and *cross_attention* (0.6951) rows are all *unweighted*, so they are directly comparable to each other. The two *gnn_only* rows are a before/after methodology pair, not two comparable conditions: 0.0112 is the original unweighted run that first exposed a label-imbalance problem, and 0.1621 is the same setup with positive-class weighting added to fix it. The edge-free *gnn_only* control (0.1599 mean-pool, 0.1689 max-pool, Section VI-C) uses this same positive-class weighting, matching the 0.1621/0.1640-family comparators

TABLE I

TASK 2, GTZAN GENRE CLASSIFICATION (150 TEST TRACKS). THE EDGE-FREE ENCODER USES A PER-NODE MLP OF MATCHED WIDTH AND DEPTH IN PLACE OF EVERY GRAPH SAGE LAYER, WITH NO EDGES; THIS CHANGES THE AGGREGATION OPERATOR AS WELL AS REMOVING TOPOLOGY, A CAVEAT DISCUSSED IN SECTION VII.

Model	Params	Macro-F1
CNN (mel-spec., no graph, no text)	–	0.7654
GraphSAGE (deployed, mean-pool)	13,066	0.5976
Edge-free, mean-pool [†]	6,922	0.6049
Edge-free, max-pool	6,922	0.6425
Exact 2-hop (fixed aggr., trained head)	6,858	0.6630

[†]Pooling-matched to the deployed GraphSAGE row above.

it is measured against, not the unweighted 0.0112 row. A positive-weighted *cross_attention* variant also exists separately in the evidence base (0.6769, lower than the unweighted 0.6951 reported here) but is not otherwise used in this paper’s comparisons; we did not selectively omit it – it simply answers a different question (does weighting change *cross_attention* specifically) than Table II does. No positive-weighted *bert_only* or *concat* run exists in the evidence base. Task 1 does not use positive-class weighting.

VI. RESULTS

A. Task 1: what can text already do?

A fine-tuned DistilBERT classifier [12] reaches macro-F1 **0.7091** (micro-F1 0.7503, AUC-PR 0.7302) on MusicCaps tag prediction from captions alone, with no audio or graph involved. Given the 69.1% lexical exposure measured in Section III, part of this strength is a direct, literal shortcut; a caption-sanitization control (target words removed, model retrained) still recovers 84.3% of this score, so the result is not purely a leakage artifact. This is not a numerical baseline the other tasks are scored against – Task 2 uses a different dataset and label space, and Task 4 is scored by Recall@K, not macro-F1 – but it is the qualitative anchor: text alone is already strong before any graph is introduced, which is the standard the rest of the paper’s “does the graph add anything” questions are asked against.

B. Task 2: does topology help audio classification?

The pooling-matched comparison – both models using mean-pool readout, the fairest single comparison in this table – has the edge-free encoder ahead by **+0.0072** macro-F1 (0.6049 vs. 0.5976), using 47% fewer parameters and no edge information at all (Figure 2). The edge-free encoder’s best result overall, at max-pool, is further ahead (+0.045), but that comparison changes both topology and pooling at once and should not be read as the clean ablation; we report it because it is the edge-free model’s genuine best result, not because it is the fairest comparison. Neither reading has the graph model ahead. A separate, repaired, pre-verified sparser graph construction (mutual k -NN, three regimes, three poolings, 8 seeds each, comparing real edges against none/shuffled/random for 27 total contrasts [10]) likewise found no training-time contrast that

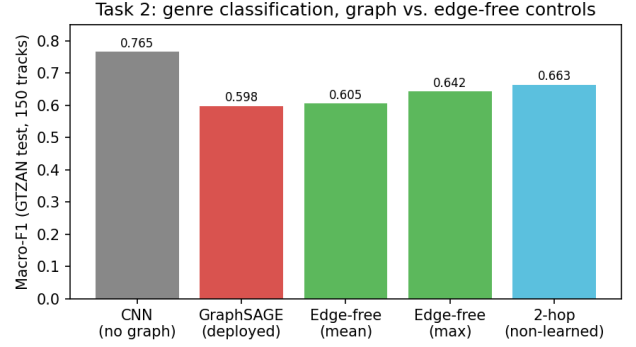


Fig. 2. Task 2: GTZAN genre classification, graph model vs. controls.

TABLE II

TASK 3, MUSICCAPS FUSION (715 TEST EXAMPLES, EXCEPT THE EDGE-FREE ROW, WHICH IS VALIDATION-ONLY – SEE NOTE BELOW).

Mode	Macro-F1	Micro-F1	AUC-PR
bert_only	0.7298	0.7647	0.7485
gnn_only (unweighted)	0.0112	0.0290	0.0879
gnn_only (pos-weighted)	0.1621	0.1839	0.1196
edge-free gnn, pos-w. (max) [†]	0.1689	–	–
concat	0.7061	0.7478	0.7367
cross_attention	0.6951	0.7398	0.7310

[†]Validation set, not test – see Section V.

reliably favored the real graph. Only 18 of these 27 (real-vs-shuffled and real-vs-random, holding the same GraphSAGE architecture fixed) are the cleaner topology-only intervention this sentence relies on, changing edge identity alone; the remaining 9 (real-vs-none) are a real-vs-edge-free comparison like this paper’s own Task 2 control, not a topology-isolating one, and are not double-counted as topology-only evidence here. Non-learned exact 2-hop aggregation also beats GraphSAGE, suggesting part of the gap is not only about topology carrying little signal, but about GraphSAGE’s learned aggregation not fully using what signal is present – a distinction we return to in Section VIII.

C. Task 3: does the graph complement text?

Text alone (0.7298) outperforms every fusion mode; neither *concat* (0.7061) nor *cross_attention* (0.6951) improves on it. The edge-free *gnn_only* control was only evaluated on validation, not test, and the pooling-matched comparison here does not repeat Task 2’s pattern: at mean-pool (matching the deployed GraphSAGE’s own pooling), the GraphSAGE-equivalent baseline reaches 0.1640 against the edge-free encoder’s 0.1599 – the graph slightly *ahead* by +0.0041. Only at max-pool does the edge-free encoder lead (0.1689). Both edge-free readings use the same positive-class weighting as the 0.1640 GraphSAGE-equivalent comparator, so this specific pair is weighting-matched, unlike the Table II rows discussed under Class Weighting (Section V). We report both honestly rather than quote only the max-pool number: this comparison, unlike Task 2’s, does not show a clean edge-free advantage

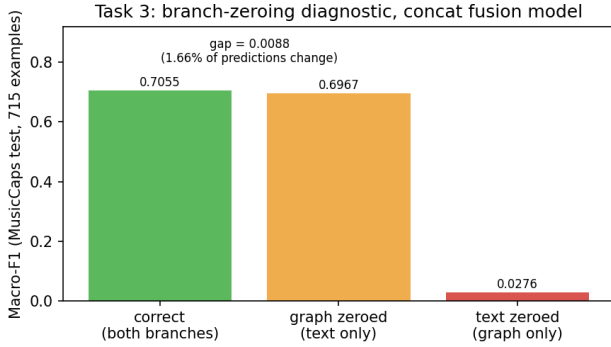


Fig. 3. Task 3: branch-zeroing diagnostic on the trained *concat* fusion model.

once pooling is held fixed, and we do not treat it as this task’s primary graph-vs-edge-free evidence in either direction – that role belongs to the branch-zeroing and complementarity diagnostics below, both computed on the real test set and unaffected by this pooling question.

Branch-zeroing (Figure 3): replacing the graph embedding with zeros inside the trained *concat* model changes only **1.66%** of individual (example, tag) predictions and costs **0.0088** macro-F1 (0.7055 $\rightarrow$ 0.6967; this 0.7055 is the branch-diagnostic run’s own re-measurement of *concat*, negligibly different from the 0.7061 in Table II above); zeroing the text branch instead costs 0.6779 macro-F1, confirming text is overwhelmingly the informative branch. This diagnostic is reproduced directly from this repository (`task3_branch_zeroing.py`), rather than only cited from an earlier, separate diagnostic script, so it is independently re-derivable here. **Complementarity**: cross-tabulating *bert_only* against *gnn_only* predictions – two separately-trained, single-branch models, not the fusion model itself (35,750 (example, tag) cells, 715 test examples $\times$ 50 tags) – using the standard, unweighted *gnn_only* checkpoint (macro-F1 0.0112 on this multi-label task) finds the graph model correct on 508 of the 1,047 cells where text is wrong (**48.5%**, an apparent “rescue rate”). Taken at face value this looks like real, if modest, complementary signal. It is not: this checkpoint’s own per-tag precision and recall (Table II’s source metrics) are exactly 0.0 for 45 of 50 tags – it predicts a tag present at all for only 5 of them – so most of its “correct” cells are true negatives reached by never predicting positive, not audio-based detection. Splitting the 508 rescued cells by label polarity (a follow-up check, `task3_complementarity_by_polarity.py`) confirms this directly: only **3 of 508** (0.6%) are the graph model actually producing a true positive where text produced a false negative; the remaining **505 of 508** (99.4%) are the graph model defaulting to “absent” while text wrongly said “present” – consistent with a checkpoint that mostly does not predict positive at all, though a true negative alone does not strictly prove no audio signal contributed to it. A separate way to see the same point: an always-predict-absent classifier

TABLE III
TASK 4, MUSICCAPS RETRIEVAL (673 TEST EXAMPLES), MEAN $\pm$ POPULATION STANDARD DEVIATION ($\text{DDOF}=0$) OVER 3 SEEDS.

Metric	Real graph	Edge-free
Audio $\rightarrow$ Text R@1	0.0124 $\pm$ 0.0019	0.0124 $\pm$ 0.0046
Audio $\rightarrow$ Text R@5	0.0570 $\pm$ 0.0062	0.0589 $\pm$ 0.0046
Audio $\rightarrow$ Text R@10	0.1025 $\pm$ 0.0053	0.1000 $\pm$ 0.0091
Text $\rightarrow$ Audio R@1	0.0099 $\pm$ 0.0031	0.0104 $\pm$ 0.0032
Text $\rightarrow$ Audio R@5	0.0515 $\pm$ 0.0037	0.0570 $\pm$ 0.0101
Text $\rightarrow$ Audio R@10	0.1030 $\pm$ 0.0125	0.1010 $\pm$ 0.0063

would, by construction, be “correct” on every one of the 509 cells where text has a false positive, a 509-of-1,047 (48.6%) rescue rate – essentially the same as the 48.5% actually observed, so the headline number does not distinguish this checkpoint from a trivial one. We withdraw the “not zero” reading an earlier draft of this diagnostic gave it. It is also outnumbered by the reverse: 1,735 cells where text is right and the graph model is wrong. This cross-tabulation is correlational – it compares two independently trained models, not an intervention on either one. Branch-zeroing above is a causal intervention in a narrower sense: it manipulates the trained fusion model’s input at inference time and measures the effect on its output, which is different from a training-time test of what a fusion model would learn if the graph branch were absent from the start (zero may also be an input the model never saw in training, so this measures reliance on the branch as currently trained, not its causal contribution under retraining). With that scope in mind, it found removing the graph costs only 0.0088 macro-F1 – small, but a genuine, measured effect on the fusion model itself, which is a different and more direct kind of evidence than the complementarity diagnostic’s correlational count. One further, more tentative data point: swapping in richer, pretrained node features (frozen CNN embeddings, trained for GTZAN genre classification, in place of chroma+MFCC) gives a modest gain on a single seed, validation only (0.1640 $\rightarrow$ 0.1846, +0.0206). This is suggestive that node-feature weakness is at least part of the story, but it rests on one run, one embedding source trained for a different dataset and label space than the one it is tested on here, and no test-set or multi-seed confirmation – too thin to treat as more than a hint.

D. Task 4: does any graph benefit survive under retrieval?

Across three training seeds, the absolute difference between the real-graph and edge-free encoders’ means ranges from 0.00000 to 0.00545 across the six reported metrics (Figure 4) – smaller, on every metric, than at least one of the two conditions’ own population standard deviations ($\text{ddof}=0$), though not always smaller than both, so “within one SD” is not a uniformly true or precise description and we do not use it. With only three runs and no paired significance or equivalence test computed, the defensible statement is that we observed no consistent advantage for either encoder in either retrieval direction, not a formal proof of equivalence. This is a more informative result than an earlier, single-seed check, which

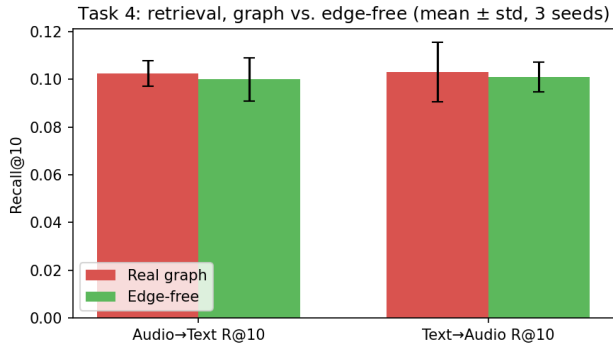


Fig. 4. Task 4: retrieval Recall@10, graph vs. edge-free encoder.

happened to show edge-free ahead on audio→text R@10 by 0.028 and graph-based ahead on text→audio R@10 by 0.015. Neither single-seed gap recurs in the three-seed rerun – both audio→text and text→audio R@10 per-seed gaps range only from -0.013 to $+0.021$ and -0.006 to $+0.010$ respectively, narrower in both directions than the corresponding earlier single-seed gap – so the earlier check’s specific magnitudes do not repeat, though three seeds is too small a basis to call this a formally established difference in variability, only a description of what these three runs happened to show. Zero-shot tag prediction (embedding tag names as text, with no supervised tag classifier; Section V) reaches macro-F1 0.146 on the single seed-42 real-graph checkpoint – not the three seeds behind Table III, and without a random-encoder control run to confirm this is above chance rather than an artifact of the threshold used, so we report the number without the stronger “confirms semantic content” claim it might otherwise support. It is, as expected, far below Task 1’s supervised classifier, for a system that never sees tag or genre labels during training (it does see which caption pairs with which clip, which is itself a form of supervision, just not a tag/genre label). In a separate, single-seed check on the real-graph encoder only, quadrupling the contrastive batch size ($32 \rightarrow 128$, more in-batch negatives per anchor) changed R@10 by at most 0.006 in either retrieval direction – suggestive evidence against batch size being the retrieval bottleneck, though this was not repeated for the edge-free encoder or across seeds, so we treat it as a secondary observation, not a controlled result on the same footing as the three-seed comparison above.

Qualitative failure review. A possible objection to exact-pair retrieval evaluation is that a “wrong” top-1 result might still be a semantically reasonable answer, just not the one official paired clip – a false negative of the evaluation protocol, not a real model error. We inspected 20 successes (true clip ranked in the top 3) and 20 failures (true clip ranked below 10th) manually, comparing each failure’s query caption against its top-1 retrieved clip’s own caption. The failures’ true-clip ranks are not close misses: median rank 166 of 673 (the 20 ranks, sorted, are 28, 47, 49, 52, 82, 86, 100, 102, 136, 150, 182, 209, 211, 230, 259, 260, 271, 289, 319, 352), ranging

from 28 to 352. Reading the caption pairs directly, the large majority describe clearly different instrumentation, genre, or mood (for example, a piano-and-propeller-sound query’s top-1 match was a Russian lullaby with vocals; a bagpipes query’s top-1 match was an electric-guitar duet with two male voices). A small number share a coarse category (both instrumental, or both heavy on reverb) without matching in specific content. We do not find evidence that exact-pair evaluation is hiding a large population of semantically-correct-but-unpaired matches in this system; most inspected failures are genuine retrieval errors. This check has real limits worth stating plainly: the 20 successes and 20 failures are the first 20 qualifying test examples in dataset order, not a random or stratified sample, and the captions were read and judged by one of the authors, without a blinded rater or a predefined scoring rubric – so this is a manual sanity check, not a formal annotation study, and we report it with that scope.

VII. CROSS-TASK ANALYSIS

Individually, each task’s result is one comparison, with its own caveats. Together, they form a pattern worth examining directly, without overstating any single piece of it.

The similarity graph is close to a deterministic function of its own node features. The similarity edges used throughout this paper are constructed as $A_{\text{sim}} = g(X)$ for a fixed threshold rule g (Section III). Because A_{sim} is a deterministic function of X and nothing else, it cannot, in principle, carry information about a downstream target Y beyond what X already carries: $I(Y; X, A_{\text{sim}}) = I(Y; X)$, a direct instance of the data-processing inequality [22]. This is a claim about Shannon information specifically – that a sufficiently powerful model, trained without limit, could not extract more from (X, A_{sim}) jointly than from X alone – not a claim that any particular finite, trained model in this paper actually reaches that limit; the experiments, not this inequality alone, are what test whether the trained models here benefit in practice. This argument covers the similarity edges specifically, not the deployed construction’s temporal edges, which encode segment order and are not recoverable from an unordered feature set. It is also not only a theoretical point here: the deployed graphs, for the two datasets this paper’s comparisons use, have mean edge density 95.4% to 97.9%, with 76.4% to 84.0% of individual graphs fully connected, and randomly rewiring the edge set reproduces almost the same graph under a different name (Section III).

The direct graph-vs-edge-free comparisons are mixed at the single-cell level, but none shows the graph clearly ahead once its own best supporting diagnostic is considered. Task 2’s pooling-matched comparison favors the edge-free encoder by $+0.0072$ macro-F1 (a larger, but pooling-confounded, $+0.045$ gap also exists and is reported separately, Section VI-B). Task 3’s standalone edge-free *gmn_only* comparison is genuinely mixed and pooling-dependent – the graph-equivalent baseline is slightly ahead at mean-pool (0.1640 vs. 0.1599) and behind at max-pool (0.1640 vs. 0.1689), both validation scores (Section VI-C) – so we do not read

this one comparison as evidence in either direction; Task 3’s more informative, test-set evidence is the branch-zeroing and complementarity diagnostics – branch-zeroing finds a small positive contribution (+0.0088 macro-F1, not zero, just small), while complementarity, once corrected for the graph-only checkpoint’s label-polarity collapse (Section VI-C), finds essentially none. Task 4’s edge-free and graph-based encoders show no reliable difference across three seeds on any metric (Section VI-D). Taken together: across the available test-set evidence, no task shows a clear graph advantage – Task 2’s width/depth-matched control has disclosed dropout and seed mismatches (Section V), Task 3’s edge-free comparison is validation-only, and Task 4 supplies the repeated test-set comparison; where a standalone comparison is mixed or pooling-dependent (Task 3), we say so rather than select the reading that fits the pattern.

The correlational diagnostic finds essentially no complementary signal; the causal diagnostic finds a small but genuine one. These are two different diagnostics, not one, and they do not need to agree in degree to agree in spirit. Complementarity (Section VI-C) is correlational: comparing two independently trained, single-branch models, the graph-only model is correct on 508 of 1,047 cells where the text-only model is wrong, a 48.5% headline “rescue rate.” This number is misleading on its own: the graph-only checkpoint has exactly 0.0 precision and recall on 45 of its 50 tags, so splitting the 508 rescued cells by label polarity shows only 3 (0.6%) are the graph model genuinely producing a true positive where text produced a false negative; the other 505 of 508 (99.4%) are the graph model defaulting to “absent” while text wrongly said “present” – an always-predict-absent classifier would rescue essentially the same 48.6% of text’s errors, so this headline number does not distinguish the checkpoint from a trivial one. Branch-zeroing is a within-checkpoint inference-time intervention, not a correlational count across two separate checkpoints: inside the actual trained fusion model, removing the graph branch costs 0.0088 macro-F1 and changes 1.66% of individual predictions – small, but a real, measured effect on the model as currently trained (Section VI-C notes what this diagnostic does and does not establish about training-time causation). The two diagnostics are not in tension: complementarity, properly read, finds only three positive-label rescues – too few to establish meaningful complementary detection; branch-zeroing finds the fusion model relies on the graph branch only slightly, not not at all. Both point to weak graph reliance, at two different levels of evidence.

Text dominance and graph weakness are separate findings, not one explanation for the other. Caption text leaks a substantial fraction of target labels directly (69.1% label-macro-averaged lexical exposure, measured for Task 1), which explains part of Task 1’s text strength on its own, independent of anything about the graph; the sanitization control that measures this precisely (Section VI) was only run for Task 1, so we do not claim the same 69.1% figure or the same sanitization result for Task 3 – Task 3’s captions are the same MusicCaps captions, so the same leakage channel plausibly

exists there too, but this was not separately measured. The graph’s weakness is established separately, by the edge-free controls, not inferred from text’s strength. Both are true at once and are reported as two distinct findings.

Dataset integrity is a genuine open question for the small comparisons, though not for the largest ones. The one confirmed leakage effect (GTZAN’s 7 cross-split exact-duplicate pairs, 3 of 150 test tracks affected, Section III) is small relative to Task 2’s CNN-vs-GraphSAGE gap (0.168) and its own larger, pooling-confounded edge-free comparison (0.045); three memorized tracks are unlikely to move either by that much. It is not demonstrably too small, however, to matter for Task 2’s smaller, pooling-matched gap (0.0072): macro-F1 can move more than a raw accuracy count suggests when errors concentrate within one genre, and a duplicated validation-split track can also affect which checkpoint is selected. We do not correct the split and rerun Task 2 in this paper – that would change every Task 2 number reported here and is left as a direct next step – so we state this honestly as an unresolved caveat on the smallest comparison specifically, not on the pattern as a whole. We should be precise about what a companion audit’s repaired factorial [10] does and does not add here: it repairs the *graph construction* (a separate density defect, unrelated to this leakage) and finds the same qualitative pattern, but it reuses the same underlying GTZAN train/val/test split, so it does *not* escape this same duplicate-leakage caveat – we do not claim it as leakage-free corroboration. What it and Task 4 do add is a different kind of independence: Task 4 uses MusicCaps (checked clean of split-level ID leakage, Section III) and a completely different task type, so at least one leg of the overall pattern does not share GTZAN’s specific leakage risk at all.

The pattern holds across three structurally different learning settings, though not via the identical measurement in each. Single-label classification (Task 2), multi-label classification with a strong second modality present (Task 3), and ranking under a contrastive objective with no genre or tag class labels, only audio-caption pairing (Task 4) are three different problems, evaluated with three different metrics, and we do not claim one uniform comparison recurs across all three. Task 2’s direct graph-vs-edge-free comparison favors edge-free. Task 3’s own direct comparison is mixed (Section VI-C); what recurs there is not “edge-free ahead” but the same underlying conclusion – little detectable graph benefit – reached via one inference-time intervention and one correlational diagnostic instead. Task 4 shows no reliable difference either way. The consistent thread across all three is the *conclusion* (no reliable graph benefit detected), not an identically-shaped comparison in each case, which is more informative than any one task alone but should be read with the caveats above (validation-only numbers and a mixed standalone result in Task 3, unresolved leakage for Task 2’s smallest comparison, two separate implementations across Tasks 1–3 versus Task 4) – a consistent signal, not four independent, airtight proofs of the same fact.

VIII. DISCUSSION

What this shows. Across three tasks with different objectives, loss functions, and evaluation metrics, we could not detect a reliable benefit from this system’s similarity-based audio graph beyond what its own node features already provide – though the specific evidence differs by task, and we do not claim it is uniform. Task 2’s pooling-matched comparison favors the edge-free encoder directly. Task 3’s own standalone graph-vs-edge-free comparison is mixed and pooling-dependent and is *not* the evidence we rely on there; its branch-zeroing and complementarity diagnostics, both on held-out test data, are what show little reliance on the graph. Task 4 shows no reliable difference across three seeds. Tasks 2 and 4 rely directly on width/depth-matched, lower-parameter edge-free controls; Task 3 also has such a control, but only at validation, so its relied-upon test-set evidence comes from branch-zeroing and complementarity instead, neither of which is an edge-free-encoder comparison. The three findings are therefore not built from one identical comparison type, and the paper says so at each step rather than only here.

What this does not show. It does not show that graph neural networks are a poor choice for music understanding in general, or that no graph construction could help this kind of system. It shows that *this specific* feature-induced similarity graph, applied to *these* node features, on *these* datasets, gave the model little it did not already have. A graph built from a genuinely external relation – one not recomputable from the same features given to each node, such as annotated chord progressions, beat structure, or section repetition – is not ruled out by this study and remains untested here.

Why this might be happening, and a genuine complication. The clearest candidate explanation is structural: a similarity threshold applied to the same features used as node content produces an adjacency matrix that is, in the information-theoretic sense, redundant with those features (Section VII). A secondary, more speculative possibility is that even the limited topology present is not fully exploited by the trained GraphSAGE model – the broader GNN literature has separately found that trained message-passing layers can under-use structure through over-smoothing [23], and that a fixed propagation step can match or exceed a trained GCN-style layer [24]. Table I’s exact 2-hop result (0.6630, beating both edge-free numbers and GraphSAGE, not the CNN baseline at 0.7654) is a genuine complication for a “topology never helps” reading, reported rather than explained away – but not as a clean ablation: the 2-hop pipeline uses full-batch Adam for a fixed 200 epochs and no dropout, while GraphSAGE’s training script does not pass its configured dropout through and so trains at the model default of 0.2 (Section V discloses this bug precisely), with mini-batches of 32 and early stopping over up to 100 epochs; both use the same nominal weight decay, 10^{-4} . These training-recipe differences, not only whether the aggregation is learned, could plausibly account for some or all of the gap, so we treat it as an exploratory data point, not evidence that isolates topology or aggregation type on

its own – the cleaner topology-only evidence this paper relies on is the same-architecture real/shuffled/ random-edge factorial from a companion audit [10] (Section VI-B, included alongside this paper as a supplementary manuscript, not only as an external citation, so its full methods and result tables are directly inspectable). We do not have an equivalent 2-hop control for Tasks 3 or 4, and this result is single-run despite sharing GraphSAGE’s documented seed (42); it is also, we found while porting its script into this repository for reproducibility (`task2_exact_2hop_control.py`), not exactly reproducible across environments – an independent rerun with identical code, seed, and data reached 0.6551, not 0.6630, plausibly due to floating-point non-associativity in the two-hop aggregation’s accumulation step. Both values still clearly exceed GraphSAGE (0.5976) and both edge-free readings (0.6049, 0.6425), so this does not change the qualitative finding, but it is a further, concrete reason to treat this result as exploratory rather than precise. We do not treat it as overturning the paper’s overall pattern – but it is a real, measured number that keeps this paper from concluding the deployed encoders extract everything the node features have to offer.

Limitations. The caption-sanitization control (69.1% lexical exposure, 84.3% retention after retraining, Section III) was run in an internal research directory outside the two codebases this paper otherwise draws from; the stored numbers were directly re-verified against their result files for this paper, but the training script itself is not included alongside this paper, a reproducibility gap disclosed here rather than left implicit. Task 2 and Task 3’s main comparisons use a single training seed each; only Task 4’s headline comparison is repeated across seeds. GTZAN carries a small, previously undocumented amount of exact-duplicate, cross-split leakage (Section III); MusicCaps was checked clean at the ID level but not at the audio-content level, so two different clips of the same underlying recording, if any exist, would not have been caught. Neither dataset’s artist-level leakage could be checked, since neither ships usable artist metadata. This paper uses the original, denser graph construction throughout (Section III); a companion audit of the same system repairs this construction and finds the same qualitative pattern on the repaired graphs [10], but the present paper’s own four-task comparisons were not rerun on that repaired construction. Retrieval evaluation here treats only the one official caption as correct for a given clip; a semantically reasonable but unpaired clip would be scored as an error under this protocol. We checked this directly on 20 retrieval failures (Section VI-D) and did not find evidence of a large hidden population of such cases – most failures were genuine mismatches on manual inspection, with the true clip ranked far outside the top 10. This check used only one checkpoint, 20 non-randomly-selected examples (the first 20 qualifying cases in test order), and one unblinded rater with no predefined rubric, so it does not rule out the effect entirely, only that it is not large enough to be obvious on a direct look.

A follow-up check that did not confirm the Task 3

finding, and why. We reran Task 3’s branch-zeroing and complementarity diagnostics in a second, independent implementation of this same architecture family – the one Task 4’s own numbers are drawn from – and did not get the same answer. There, removing the graph branch costs 0.12–0.17 macro-F1, not 0.0088, and the complementarity check finds 33–47% genuine rescue, not 0.6%. Both readings are real measurements, not a mistake in either one, so this needed an explanation rather than a choice of which number to believe. We found one: that implementation trains all four fusion modes, including *gnn_only*, with positive-class weighting, while the *gnn_only* checkpoint behind this section’s complementarity numbers was trained without it and had collapsed to predicting a tag present for only 5 of its 50 tags. We checked this was not simply about how long training ran, by retraining the second implementation’s *concat* and *gnn_only* models for the same short number of epochs used before – the larger effect was still there. We are reporting this as an open question about how sensitive this particular finding is to a class-weighting choice, not as a reason to doubt the branch-zeroing and complementarity numbers reported above, and not as evidence that the graph actually does help once weighted correctly – we did not attempt to settle which weighting choice is the right one for this diagnostic, and leave that for further work.

Why richer relations might behave differently. A relation that is not a function of the node features – for example, which segments belong to the same repeated section, or which two chords typically follow one another – would fall outside the redundancy argument in Section VII by construction, since it encodes something the feature vectors alone cannot recover. Testing this is a natural next step, not attempted here beyond the informal argument above.

IX. CONCLUSION

For similarity edges, Shannon redundancy with the node features follows analytically from the construction, not from any experiment. What we tested, rather than assumed, is the practical question – whether this similarity-based audio graph nevertheless helps a real, finite model – across four tasks spanning text-only prediction, graph-only classification, multimodal fusion, and cross-modal retrieval. Tasks 2 and 4 include a direct, test-set graph-vs-edge-free comparison; Task 3’s own such comparison is validation-only, so its evidence instead comes from two test-set diagnostics: branch-zeroing inside the trained fusion model, and a cross-tabulation of independently-trained single-branch checkpoints. No *learned-aggregation* graph-based model ever showed a clear advantage over its edge-free alternative, and it trailed in Task 2’s pooling-matched reading – a single-run difference, with GTZAN’s small unresolved leakage caveat and no uncertainty estimate, so we call this a trailing result, not a statistically established loss.

One qualifier matters here. Task 2’s non-learned exact 2-hop control, whose aggregation is fixed rather than trained, does clearly beat every edge-free number – a genuine complication,

discussed in Section VIII, not swept into “graph-based model” for convenience. Task 3’s own standalone comparison was mixed and inconclusive on its own; the evidence for little graph reliance there instead comes from two diagnostics on the trained fusion model and its parts – branch-zeroing (an inference-time intervention directly on the fusion model) and complementarity (correlational, comparing two separately-trained single-branch models) – not from that standalone number. Task 4 showed no consistent advantage either way across three seeds; with only three runs and no formal equivalence test, that is an absence of detected difference, not proof of none.

We report the resulting conclusion as the paper’s central finding: no task in this study showed a clearly established graph advantage, though the supporting evidence and its strength differ by task and are stated precisely rather than smoothed into one uniform result. Presence of a graph branch in a music-understanding pipeline is not the same as demonstrated reliance on it – the two should be checked separately, not assumed to go together.

REFERENCES

- [1] J. Foote, “Automatic audio segmentation using a measure of audio novelty,” in *Proceedings of the IEEE International Conference on Multimedia and Expo (ICME)*, 2000.
- [2] T. N. Kipf and M. Welling, “Semi-supervised classification with graph convolutional networks,” in *Proceedings of the 5th International Conference on Learning Representations (ICLR)*, 2017.
- [3] P. Veličković, G. Cucurull, A. Casanova, A. Romero, P. Liò, and Y. Bengio, “Graph attention networks,” in *Proceedings of the 6th International Conference on Learning Representations (ICLR)*, 2018.
- [4] K. Xu, W. Hu, J. Leskovec, and S. Jegelka, “How powerful are graph neural networks?” in *Proceedings of the 7th International Conference on Learning Representations (ICLR)*, 2019.
- [5] W. L. Hamilton, R. Ying, and J. Leskovec, “Inductive representation learning on large graphs,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2017.
- [6] F. Errica, M. Podda, D. Bacciu, and A. Micheli, “A fair comparison of graph neural networks for graph classification,” in *Proceedings of the 8th International Conference on Learning Representations (ICLR)*, 2020.
- [7] O. Shchur, M. Mumme, A. Bojchevski, and S. Günnemann, “Pitfalls of graph neural network evaluation,” *arXiv preprint arXiv:1811.05868*, 2018.
- [8] G. Tzanetakis and P. Cook, “Musical genre classification of audio signals,” *IEEE Transactions on Speech and Audio Processing*, vol. 10, no. 5, pp. 293–302, 2002.
- [9] A. Agostinelli, T. I. Denk, Z. Borsos, J. Engel, M. Verzett, A. Caillon, Q. Huang, A. Jansen, A. Roberts, M. Tagliasacchi, M. Sharifi, N. Zeghidour, and C. Frank, “MusicLM: Generating music from text,” *arXiv preprint arXiv:2301.11325*, 2023.
- [10] Anonymous, “Graph presence is not graph dependence: A controlled audit of structural utility in multimodal music understanding,” 2026, companion manuscript, same underlying system, repaired sparse graph construction; included in this submission’s supplementary materials.
- [11] B. L. Sturm, “The GTZAN dataset: Its contents, its faults, their effects on evaluation, and its future use,” *arXiv preprint arXiv:1306.1461*, 2013.
- [12] V. Sanh, L. Debut, J. Chaumond, and T. Wolf, “DistilBERT, a distilled version of BERT: Smaller, faster, cheaper and lighter,” *arXiv preprint arXiv:1910.01108*, 2019.
- [13] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, “BERT: Pre-training of deep bidirectional transformers for language understanding,” in *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics (NAACL-HLT)*, 2019, pp. 4171–4186.

- [14] K. Choi, G. Fazekas, and M. Sandler, "Automatic tagging using deep convolutional neural networks," in *Proceedings of the 17th International Society for Music Information Retrieval Conference (ISMIR)*, 2016, pp. 805–811.
- [15] M. Won, A. Ferraro, D. Bogdanov, and X. Serra, "Evaluation of cnn-based automatic music tagging models," *arXiv preprint arXiv:2006.00751*, 2020.
- [16] A. Radford, J. W. Kim, C. Hallacy, A. Ramesh, G. Goh, S. Agarwal, G. Sastry, A. Askell, P. Mishkin, J. Clark, G. Krueger, and I. Sutskever, "Learning transferable visual models from natural language supervision," in *Proceedings of the 38th International Conference on Machine Learning (ICML)*, 2021.
- [17] A. van den Oord, Y. Li, and O. Vinyals, "Representation learning with contrastive predictive coding," *arXiv preprint arXiv:1807.03748*, 2018.
- [18] Q. Huang, A. Jansen, J. Lee, R. Ganti, J. Y. Li, and D. P. W. Ellis, "MuLan: A joint embedding of music audio and natural language," in *Proceedings of the 23rd International Society for Music Information Retrieval Conference (ISMIR)*, 2022.
- [19] I. Manco, E. Benetos, E. Quenton, and G. Fazekas, "Contrastive audio-language learning for music," in *Proceedings of the 23rd International Society for Music Information Retrieval Conference (ISMIR)*, 2022.
- [20] B. Elizalde, S. Deshmukh, M. Al Ismail, and H. Wang, "CLAP: Learning audio concepts from natural language supervision," in *Proceedings of the IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, 2023.
- [21] S. Doh, M. Won, K. Choi, and J. Nam, "Toward universal text-to-music retrieval," in *Proceedings of the IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, 2023.
- [22] T. M. Cover and J. A. Thomas, *Elements of Information Theory*, 2nd ed. Wiley, 2006.
- [23] Q. Li, Z. Han, and X.-M. Wu, "Deeper insights into graph convolutional networks for semi-supervised learning," in *Proceedings of the 32nd AAAI Conference on Artificial Intelligence*, 2018, pp. 3538–3545.
- [24] F. Wu, T. Zhang, A. H. d. Souza Jr., C. Fifty, T. Yu, and K. Q. Weinberger, "Simplifying graph convolutional networks," in *Proceedings of the 36th International Conference on Machine Learning (ICML)*, 2019.